TECHNICAL UNIVERSITY OF SOFIA

FACULTY OF ELECTRONIC ENGINEERING AND TECHNOLOGIES

COURSE PROJECT

Control of a DC Motor’s Direction of Rotation
Using Push Buttons and Interrupts

Course: Microprocessor Systems Engineering

Prepared by: Preslav Dimitrov

Student ID: 941223057

Supervisor: Eng. Nikolay Kadiev, M.Sc.

Sofia, Bulgaria · 3 April 2026

Contents

1. Literature Review

2. Component Selection

2.1 Microcontroller

2.1.1 Functional Allocation

2.1.2 Pin Selection

2.2 Motor

2.3 H-Bridge

2.3.1 Basic Architecture

2.3.2 H-Bridge Selection

2.4 Power Supply Selection

3. Block Diagram

3.1 Explanation of Blocks and Connections

4. Circuit Schematic

4.1 Explanation of the Circuit Schematic

5. Source Code

5.1 Code Flowchart

5.2 Program Code Description

6. Conclusion

7. References

Project Requirements

The project is based on the following requirements:

• Microcontroller selected according to the project brief.

• Use of GPIO and PWM.

• Use of a 5 V DC motor.

• Use of three push buttons: Start/Stop, Left, and Right.

• Use of interrupts.

1. Literature Review

The aim of this project is to develop a system for controlling the direction of rotation of a DC motor using push buttons and interrupts in a microcontroller-based system. This involves working with GPIO and PWM interfaces, writing a control-program algorithm, and designing a circuit schematic containing modules operating at different voltages.

The device allows the direction of an electric motor to be controlled according to commands entered by the user. These commands are provided through three push buttons: Start/Stop, Left, and Right.

A device of this type can be used in everyday applications, including kitchen appliances, electric blinds and garage doors, children's toys, small robots, and fans. Industrial applications include conveyor belts, CNC machines, robots, and other equipment.

Similar technologies are already available, including Arduino motor shields, industrial motor drivers, and PLC-based control.

This project is implemented using a microcontroller, an electric motor, three push buttons, and an H-bridge to control the motor.

2. Component Selection

2.1 Microcontroller

The selected microcontroller is the STM32F103C8T6.

Reasons for selection:

• 3.3 V logic.

• Hardware PWM support.

• External interrupts available on almost every GPIO pin.

2.1.1 Functional Allocation

The system requires:

• Three inputs for the push buttons.

• Three outputs for the H-bridge: two direction-control signals and one PWM signal for speed.

• Power supply.

• Ground.

2.1.2 Pin Selection

• **PWM output: PA8 – TIM1_CH1.** Hardware PWM; an advanced timer suitable for power control.

• **Direction control: PB0 – GPIO Output; PB1 – GPIO Output.** These pins determine the direction of rotation.

• **Start/Stop button with interrupt: PA0 – EXTI0.** Pressing the button generates an interrupt.

• **Left/Right buttons: PA1 – GPIO Input; PA2 – GPIO Input.** These buttons control rotation in either direction.

• **Power:** VDD = 3.3 V; VSS = GND; NRST = reset button; BOOT0 connected to GND.

2.2 Motor

• 5 V DC motor.

• Nominal current: 300–500 mA.

• Starting current: up to 1 A.

2.3 H-Bridge

2.3.1 Basic Architecture

• Four power transistors (MOSFETs).

• Two control signals from the microcontroller.

• One PWM signal.

2.3.2 H-Bridge Selection

The L293D is a motor driver containing two H-bridges; one of them is used in this project.

Reasons for selecting it:

• Easy control from the microcontroller.

• Built-in protection features.

• Reliable operation.

• Supports motor supply voltages from 4.5 V to 36 V; this project requires 5 V.

2.4 Power Supply Selection

The project requires a 5 V supply for the motor driver and DC motor, while the microcontroller and its peripherals require 3.3 V. A practical solution is to use USB power with a voltage regulator.

USB provides 5 V, making it suitable for powering the motor and its driver. A linear voltage regulator converts the USB 5 V supply to 3.3 V for the microcontroller and its peripherals. The selected regulator is the AMS1117-3.3.

3. Block Diagram

The source document includes a block diagram illustrating connections between the microcontroller, power supply, three push buttons, H-bridge, and motor. The diagram's graphical elements could not be reproduced from the extracted text; the blocks are described below.

3.1 Explanation of Blocks and Connections

• **MCU STM32F103C8T6:** the microcontroller.

• **Power supply:** provides power to the system.

• **Button Right:** a push button connected to an MCU GPIO pin to change motor direction.

• **Button Left:** a push button connected to an MCU GPIO pin to change motor direction.

• **Button Start/Stop:** a push button connected to an MCU GPIO pin that starts and stops the DC motor.

• **H-Bridge:** an electronic circuit that allows the motor's direction of rotation to be controlled.

• **Motor:** the DC motor.

4. Circuit Schematic

The source document states that a separate PDF file of the circuit schematic was attached to make it easier to inspect. That separate schematic file was not included with the uploaded document.

4.1 Explanation of the Circuit Schematic

USB_B_Micro is the micro-USB connector used to supply 5 V to the motor and its driver; this voltage is also converted to 3.3 V to power the rest of the circuit.

J2 is a controller connector that allows testing or powering external 3.3 V peripherals. AMS1117-3.3 is the linear regulator that converts 5 V to 3.3 V. Capacitors C16 and C17 filter the circuit's input and output voltages.

Capacitors C4 through C7 filter the supply voltage for the microcontroller's main digital section. They are included according to the manufacturer's recommendations, with one capacitor per pin. Pin VDDA supplies the microcontroller's analogue section. A ferrite bead, FB1, reduces high-frequency noise on this supply; C8 and C9 provide additional filtering.

The STM32F103C8T6 has an internal clock oscillator, but this design uses an external crystal, Y1, with capacitors C12 and C13 to generate the microcontroller clock. The SW_SPDT switch and R4 are used to start the microcontroller. C11 is used for resetting the microcontroller.

U2 is the motor driver, and one half of the driver is used. Capacitors C14 and C15 filter the motor-driver supply voltage. M1 is the motor, and D1 through D4 are protection diodes.

5. Source Code

The source code below is reproduced from the uploaded project. Program logic and identifiers are retained; the Bulgarian speed comment has been translated into English.

```
**************************************************************
************
   *  @attention    *    *  <h2><center>&copy;  Copyright  (c)  2026  STMicroelectronics.    *  All  rights
    reserved.</center></h2>    *    *  This  software  component  is  licensed  by  ST  under  BSD  3-Clause
    license,    *  the  "License";  You  may  not  use  this  file  except  in  compliance  with  the    *
    License.  You  may  obtain  a  copy  of  the  License  at:    *
    opensource.org/licenses/BSD-3-Clause    *
**************************************************************
************
   */  /*  USER  CODE  END  Header  */  /*  Includes
    ------------------------------------------------------------*/  #include  "main.h"   /*  Private
    includes  ----------------------------------------------------------*/  /*  USER  CODE  BEGIN  Includes
    */   /*  USER  CODE  END  Includes  */

/*  Private  typedef  -----------------------------------------------------------*/  /*  USER  CODE  BEGIN
    PTD  */   /*  USER  CODE  END  PTD  */   /*  Private  define
    ------------------------------------------------------------*/  /*  USER  CODE  BEGIN  PD  */  /*  USER
    CODE  END  PD  */   /*  Private  macro  -------------------------------------------------------------*/
    /*  USER  CODE  BEGIN  PM  */   /*  USER  CODE  END  PM  */   /*  Private  variables
    ---------------------------------------------------------*/  TIM_HandleTypeDef  htim1;   PCD_HandleTypeDef
      hpcd_USB_FS;   /*  USER  CODE  BEGIN  PV  */  volatile  uint8_t  motor_enabled  =  0;  volatile  uint8_t
      direction  =  0;  /*  USER  CODE  END  PV  */

/*  Private  function  prototypes  -----------------------------------------------*/  void
    SystemClock_Config(void);  static  void  MX_GPIO_Init(void);  static  void  MX_USB_PCD_Init(void);  static
      void  MX_TIM1_Init(void);  /*  USER  CODE  BEGIN  PFP  */   /*  USER  CODE  END  PFP  */   /*  Private
    user  code  ---------------------------------------------------------*/  /*  USER  CODE  BEGIN  0  */   /*
      USER  CODE  END  0  */   /**    *  @brief   The  application  entry  point.    *  @retval  int    */
    int  main(void)  {    /*  USER  CODE  BEGIN  1  */     /*  USER  CODE  END  1  */

  /*  MCU  Configuration--------------------------------------------------------*/     /*  Reset  of  all
    peripherals,  Initializes  the  Flash  interface  and  the  Systick.  */    HAL_Init();     /*  USER  CODE
      BEGIN  Init  */     /*  USER  CODE  END  Init  */     /*  Configure  the  system  clock  */
    SystemClock_Config();     /*  USER  CODE  BEGIN  SysInit  */     /*  USER  CODE  END  SysInit  */     /*
    Initialize  all  configured  peripherals  */    MX_GPIO_Init();    MX_USB_PCD_Init();    MX_TIM1_Init();
       /*  USER  CODE  BEGIN  2  */    HAL_TIM_PWM_Start(&htim1,  TIM_CHANNEL_1);    /*  USER  CODE  END  2
    */

  /*  Infinite  loop  */    /*  USER  CODE  BEGIN  WHILE  */    while  (1)    {     if(motor_enabled)
              {              //  PWM  ~70%  ■■■■■■■■              __HAL_TIM_SET_COMPARE(&htim1,
    TIM_CHANNEL_1,  700);              if(direction  ==  1)  //  LEFT              {
    HAL_GPIO_WritePin(GPIOB,  GPIO_PIN_0,  GPIO_PIN_SET);                  HAL_GPIO_WritePin(GPIOB,
    GPIO_PIN_1,
GPIO_PIN_RESET);
             }              else  if(direction  ==  2)  //  RIGHT              {
    HAL_GPIO_WritePin(GPIOB,  GPIO_PIN_0,
GPIO_PIN_RESET);
                HAL_GPIO_WritePin(GPIOB,  GPIO_PIN_1,  GPIO_PIN_SET);              }          }
    else          {              //  Stop  motor

            __HAL_TIM_SET_COMPARE(&htim1,  TIM_CHANNEL_1,  0);              HAL_GPIO_WritePin(GPIOB,
    GPIO_PIN_0,  GPIO_PIN_RESET);              HAL_GPIO_WritePin(GPIOB,  GPIO_PIN_1,  GPIO_PIN_RESET);
             }      /*  USER  CODE  END  WHILE  */      /*  USER  CODE  BEGIN  3  */    }    /*  USER  CODE
      END  3  */  }   /**    *  @brief  System  Clock  Configuration    *  @retval  None    */  void
    SystemClock_Config(void)  {    RCC_OscInitTypeDef  RCC_OscInitStruct  =  {0};    RCC_ClkInitTypeDef
    RCC_ClkInitStruct  =  {0};    RCC_PeriphCLKInitTypeDef  PeriphClkInit  =  {0};     /**  Initializes  the
    RCC  Oscillators  according  to  the  specified  parameters    *  in  the  RCC_OscInitTypeDef  structure.
        */

  RCC_OscInitStruct.OscillatorType  =  RCC_OSCILLATORTYPE_HSE;    RCC_OscInitStruct.HSEState  =  RCC_HSE_ON;
       RCC_OscInitStruct.HSEPredivValue  =  RCC_HSE_PREDIV_DIV2;    RCC_OscInitStruct.HSIState  =
    RCC_HSI_ON;    RCC_OscInitStruct.PLL.PLLState  =  RCC_PLL_ON;    RCC_OscInitStruct.PLL.PLLSource  =
    RCC_PLLSOURCE_HSE;    RCC_OscInitStruct.PLL.PLLMUL  =  RCC_PLL_MUL9;    if
    (HAL_RCC_OscConfig(&RCC_OscInitStruct)  !=  HAL_OK)    {      Error_Handler();    }    /**  Initializes
    the  CPU,  AHB  and  APB  buses  clocks    */    RCC_ClkInitStruct.ClockType  =
RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
                          |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;    RCC_ClkInitStruct.SYSCLKSource  =
      RCC_SYSCLKSOURCE_PLLCLK;    RCC_ClkInitStruct.AHBCLKDivider  =  RCC_SYSCLK_DIV1;
    RCC_ClkInitStruct.APB1CLKDivider  =  RCC_HCLK_DIV2;    RCC_ClkInitStruct.APB2CLKDivider  =  RCC_HCLK_DIV1;
         if  (HAL_RCC_ClockConfig(&RCC_ClkInitStruct,  FLASH_LATENCY_2)  !=
HAL_OK)
   {      Error_Handler();

  }    PeriphClkInit.PeriphClockSelection  =  RCC_PERIPHCLK_USB;    PeriphClkInit.UsbClockSelection  =
    RCC_USBCLKSOURCE_PLL_DIV1_5;    if  (HAL_RCCEx_PeriphCLKConfig(&PeriphClkInit)  !=  HAL_OK)    {
    Error_Handler();    }  }   /**    *  @brief  TIM1  Initialization  Function    *  @param  None    *
    @retval  None    */  static  void  MX_TIM1_Init(void)  {     /*  USER  CODE  BEGIN  TIM1_Init  0  */
```

```
    /*  USER  CODE  END  TIM1_Init  0  */     TIM_MasterConfigTypeDef  sMasterConfig  =  {0};
    TIM_OC_InitTypeDef  sConfigOC  =  {0};    TIM_BreakDeadTimeConfigTypeDef  sBreakDeadTimeConfig  =  {0};

   /*  USER  CODE  BEGIN  TIM1_Init  1  */     /*  USER  CODE  END  TIM1_Init  1  */    htim1.Instance  =
    TIM1;    htim1.Init.Prescaler  =  72-1;    htim1.Init.CounterMode  =  TIM_COUNTERMODE_UP;
    htim1.Init.Period  =  1000-1;    htim1.Init.ClockDivision  =  TIM_CLOCKDIVISION_DIV1;
    htim1.Init.RepetitionCounter  =  0;    htim1.Init.AutoReloadPreload  =  TIM_AUTORELOAD_PRELOAD_DISABLE;
        if  (HAL_TIM_PWM_Init(&htim1)  !=  HAL_OK)    {      Error_Handler();    }
    sMasterConfig.MasterOutputTrigger  =  TIM_TRGO_RESET;    sMasterConfig.MasterSlaveMode  =
    TIM_MASTERSLAVEMODE_DISABLE;    if  (HAL_TIMEx_MasterConfigSynchronization(&htim1,  &sMasterConfig)  !=
HAL_OK)
   {      Error_Handler();    }    sConfigOC.OCMode  =  TIM_OCMODE_PWM1;    sConfigOC.Pulse  =  0;
    sConfigOC.OCPolarity  =  TIM_OCPOLARITY_HIGH;

  sConfigOC.OCNPolarity  =  TIM_OCNPOLARITY_HIGH;    sConfigOC.OCFastMode  =  TIM_OCFAST_DISABLE;
    sConfigOC.OCIdleState  =  TIM_OCIDLESTATE_RESET;    sConfigOC.OCNIdleState  =  TIM_OCNIDLESTATE_RESET;
    if  (HAL_TIM_PWM_ConfigChannel(&htim1,  &sConfigOC,
TIM_CHANNEL_1)

!=

HAL_OK)
   {      Error_Handler();    }    sBreakDeadTimeConfig.OffStateRunMode  =  TIM_OSSR_DISABLE;
    sBreakDeadTimeConfig.OffStateIDLEMode  =  TIM_OSSI_DISABLE;    sBreakDeadTimeConfig.LockLevel  =
    TIM_LOCKLEVEL_OFF;    sBreakDeadTimeConfig.DeadTime  =  0;    sBreakDeadTimeConfig.BreakState  =
    TIM_BREAK_DISABLE;    sBreakDeadTimeConfig.BreakPolarity  =  TIM_BREAKPOLARITY_HIGH;
    sBreakDeadTimeConfig.AutomaticOutput  =
TIM_AUTOMATICOUTPUT_DISABLE;
   if  (HAL_TIMEx_ConfigBreakDeadTime(&htim1,  &sBreakDeadTimeConfig)  !=
HAL_OK)
   {      Error_Handler();    }    /*  USER  CODE  BEGIN  TIM1_Init  2  */     /*  USER  CODE  END  TIM1_Init
      2  */

  HAL_TIM_MspPostInit(&htim1);   }   /**    * @brief  USB  Initialization  Function    * @param  None    *
    @retval  None    */  static  void  MX_USB_PCD_Init(void)  {     /*  USER  CODE  BEGIN  USB_Init  0  */
         /*  USER  CODE  END  USB_Init  0  */     /*  USER  CODE  BEGIN  USB_Init  1  */     /*  USER  CODE
    END  USB_Init  1  */    hpcd_USB_FS.Instance  =  USB;    hpcd_USB_FS.Init.dev_endpoints  =  8;
    hpcd_USB_FS.Init.speed  =  PCD_SPEED_FULL;    hpcd_USB_FS.Init.low_power_enable  =  DISABLE;
    hpcd_USB_FS.Init.lpm_enable  =  DISABLE;

  hpcd_USB_FS.Init.battery_charging_enable  =  DISABLE;    if  (HAL_PCD_Init(&hpcd_USB_FS)  !=  HAL_OK)    {
         Error_Handler();    }    /*  USER  CODE  BEGIN  USB_Init  2  */     /*  USER  CODE  END  USB_Init  2
      */   }   /**    * @brief  GPIO  Initialization  Function    * @param  None    * @retval  None    */
    static  void  MX_GPIO_Init(void)  {    GPIO_InitTypeDef  GPIO_InitStruct  =  {0};     /*  GPIO  Ports
    Clock  Enable  */    __HAL_RCC_GPIOD_CLK_ENABLE();    __HAL_RCC_GPIOA_CLK_ENABLE();
    __HAL_RCC_GPIOB_CLK_ENABLE();

   /*Configure  GPIO  pin  Output  Level  */    HAL_GPIO_WritePin(GPIOB,  Left_output_Pin|Right_output_Pin,
GPIO_PIN_RESET);
    /*Configure  GPIO  pins  :  PA0  PA1  PA2  */    GPIO_InitStruct.Pin  =  GPIO_PIN_0|GPIO_PIN_1|GPIO_PIN_2;
        GPIO_InitStruct.Mode  =  GPIO_MODE_IT_FALLING;    GPIO_InitStruct.Pull  =  GPIO_PULLUP;
    HAL_GPIO_Init(GPIOA,  &GPIO_InitStruct);     /*Configure  GPIO  pins  :  Left_output_Pin  Right_output_Pin
      */    GPIO_InitStruct.Pin  =  Left_output_Pin|Right_output_Pin;    GPIO_InitStruct.Mode  =
    GPIO_MODE_OUTPUT_PP;    GPIO_InitStruct.Pull  =  GPIO_NOPULL;    GPIO_InitStruct.Speed  =
    GPIO_SPEED_FREQ_LOW;    HAL_GPIO_Init(GPIOB,  &GPIO_InitStruct);   }   /*  USER  CODE  BEGIN  4  */  void
      HAL_GPIO_EXTI_Callback(uint16_t  GPIO_Pin)  {   HAL_Delay(50);  //  debounce      if(GPIO_Pin  ==
    GPIO_PIN_0)  //  Start/Stop

    {         motor_enabled  =  !motor_enabled;      }      if(GPIO_Pin  ==  GPIO_PIN_1)  //  Left      {
             direction  =  1;      }      if(GPIO_Pin  ==  GPIO_PIN_2)  //  Right      {         direction
    =  2;      }  }  /*  USER  CODE  END  4  */   /**    * @brief   This  function  is  executed  in  case
    of  error  occurrence.    * @retval  None    */  void  Error_Handler(void)  {    /*  USER  CODE  BEGIN
    Error_Handler_Debug  */    /*  User  can  add  his  own  implementation  to  report  the  HAL  error
    return  state  */     /*  USER  CODE  END  Error_Handler_Debug  */

}   #ifdef   USE_FULL_ASSERT  /**    * @brief   Reports  the  name  of  the  source  file  and  the  source
    line  number    *          where  the  assert_param  error  has  occurred.    * @param   file:  pointer
    to  the  source  file  name    * @param   line:  assert_param  error  line  source  number    * @retval
      None    */  void  assert_failed(uint8_t  *file,  uint32_t  line)  {    /*  USER  CODE  BEGIN  6  */
    /*  User  can  add  his  own  implementation  to  report  the  file  name  and  line  number,       tex:
    printf("Wrong  parameters  value:  file  %s  on  line  %d\r\n",  file,  line)  */    /*  USER  CODE  END
    6  */  }  #endif  /*  USE_FULL_ASSERT  */   /************************  (C)  COPYRIGHT  STMicroelectronics
      *****END
OF

FILE****/
```

5.1 Code Flowchart

The source document contains a flowchart on this page, but its graphical content was not available as extractable text. The program sequence is described in the next section.

5.2 Program Code Description

The program controls an electric motor using an STM32-series microcontroller. Pulse-width modulation (PWM) regulates speed, while digital outputs set the direction of rotation. Input commands are provided through push buttons connected to GPIO pins configured to operate using interrupts.

After startup, the program initializes the required peripherals: GPIO ports, timer TIM1, and the USB module. The timer is configured in PWM mode to generate the signal used to control motor speed.

The main program logic runs in an infinite loop. It checks the motor_enabled variable, which determines whether the motor should be running. If the motor is enabled, a PWM value corresponding to approximately 70% duty cycle is set. The direction variable determines the logic states of two GPIO pins, setting the motor’s direction of rotation (left or right).

If the motor is not enabled, the PWM value is set to zero, stopping the motor, and the output pins are reset.

• **HAL_TIM_PWM_Start()** starts PWM generation on the specified timer and channel.

• **__HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1, value)** sets the capture/compare register (CCR) value, which determines the PWM duty cycle.

• **HAL_GPIO_WritePin(GPIOB, GPIO_PIN_X, GPIO_PIN_SET/RESET)** sets a GPIO pin's logic state: SET = logic 1 (HIGH); RESET = logic 0 (LOW).

• **void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)** is called automatically when an external GPIO interrupt occurs. It identifies the button pressed and executes the corresponding logic.

• **HAL_Delay(50)** pauses program execution for a specified time and is used for button debouncing.

6. Conclusion

The aim of this course project was to design and implement a system for controlling an electric motor's direction using a microcontroller, push buttons, PWM, and interrupts.

A block diagram was developed, the system components were selected, and a complete circuit schematic was designed. The source code provides motor control, while interrupts allow the system to respond quickly to input signals.

In conclusion, the developed system meets the project's stated objectives and demonstrates effective control of a DC motor.

7. References

https://www.st.com/resource/en/datasheet/stm32f103c8.pdf

https://www.st.com/en/development-tools/stm32cubeide.html#overview

https://www.youtube.com/@stm32world